LAPORAN TUGAS KELOMPOK

Mata Kuliah Sistem Operasi (USA-WP2260110)

Analisis dan Penanganan Deadlock pada Sumber Daya Kebun Pintar

Keterangan	Isi
Kode dan judul tugas	K-3 Analisis dan Penanganan Deadlock
Kelas / tema	IF-3C — IoT dan Smart Farming
Nama	Dendy Zainal Mutaqien
NIM	250660121068
Anggota kelompok	3 orang — Dendy Zainal Mutaqien, R.Sangga Buana, M.Azkal Azkiya (rincian di Lampiran D)
Nama host	INFINIX-ElementaryOS
Minggu / tanggal kumpul	Minggu ke-7 / 2 November 2026
Kepingan puzzle	Kepingan #7 → Level 5 Puzzle UTS
Dosen pengampu	Muhammad Agreindra Helmiawan, S.Kom., M.T

Program Studi S1 Informatika
Fakultas Teknologi Informasi · Universitas Sebelas April
2026

Daftar Isi

1. Pendahuluan..... 3
1.1 Latar belakang kasus..... 3
1.2 Tujuan tugas..... 3
1.3 Ruang lingkup..... 3
2. Dasar Teori Singkat..... 3
3. Lingkungan Praktik..... 4
4. Langkah Pengerjaan..... 5
4.1 Langkah 1: Menyiapkan matriks masukan..... 5
4.2 Langkah 2: Menghitung Need dan safe sequence..... 6
4.3 Langkah 3: Menggambar resource-allocation graph..... 9
4.4 Langkah 4: Menguji permintaan sumber daya..... 12
5. Hasil dan Analisis..... 12
5.1 Hasil..... 12
5.2 Analisis..... 14
5.2.1 Empat kondisi Coffman pada kasus ini..... 14
5.2.2 Mengapa sistem dinyatakan aman..... 15
5.2.3 Mengapa satu permintaan ditolak..... 15
5.3 Rekomendasi..... 15
5.4 Catatan keterbatasan..... 16
6. Kesimpulan..... 16
7. Kepingan Puzzle..... 17
Daftar Pustaka..... 17
Lampiran A. Kode Program dan Log..... 18
Lampiran B. Pernyataan Penggunaan AI Generatif..... 23
Lampiran C. Daftar Periksa Sebelum Mengumpulkan..... 23
Swa-nilai berdasarkan kriteria lembar tugas..... 24
Lampiran D. Lembar Kontribusi Anggota..... 25

1. Pendahuluan

1.1 Latar belakang kasus

Sistem irigasi otomatis pada kebun pintar bukan satu program, melainkan beberapa tugas kontrol yang berjalan bersamaan di satu mikrokontroler: penjadwal irigasi, kontrol pompa, kontrol zona lahan, pembaca kelembapan, dan pencatat telemetri. Kelima tugas itu memakai tiga sumber daya yang sama: pompa air, katup irigasi, dan bus sensor I2C.

Ketiga sumber daya itu tidak dapat dibagi seenaknya. Satu pompa tidak boleh dikendalikan dua tugas sekaligus, dan satu transaksi I2C hanya boleh dilayani satu perangkat pada satu waktu. Karena itu setiap tugas harus meminta izin sebelum memakainya, dan menunggu bila sedang dipakai tugas lain. Di sinilah bahaya muncul: kalau tugas A memegang pompa lalu menunggu katup, sementara tugas B memegang katup lalu menunggu pompa, keduanya menunggu selamanya. Keadaan itu disebut deadlock, dan pada sistem irigasi akibatnya bukan sekadar program berhenti — tanaman bisa kekeringan atau lahan kebanjiran karena pompa tidak pernah dimatikan.

Laporan ini menganalisis kemungkinan itu dengan tiga pendekatan yang diminta lembar tugas: menggambar resource-allocation graph, menerapkan algoritma Banker untuk menentukan safe state, lalu merancang strategi pencegahan yang realistis untuk perangkat IoT.

1.2 Tujuan tugas

Capaian pembelajaran yang diukur adalah Sub-CPMK-7 (CPMK061, CPMK063, CPMK081, CPMK102, CPMK112). Tujuannya:

- **Pertama,** menggambar resource-allocation graph kasus kebun pintar dan mengidentifikasi empat kondisi Coffman yang terpenuhi;
- **Kedua,** menerapkan algoritma Banker untuk menentukan apakah sistem berada pada safe state, termasuk menguji permintaan sumber daya yang masuk;
- **Ketiga,** merancang strategi pencegahan deadlock yang masuk akal untuk mikrokontroler dengan memori dan prosesor terbatas.

1.3 Ruang lingkup

Analisis memakai 5 tugas kontrol dan 3 jenis sumber daya dengan total instans (10, 5, 7). Seluruh perhitungan dijalankan oleh banker.py pada Python 3.12.3 di mesin elementary OS 8, lalu diverifikasi ulang terhadap langkah manual pada Bagian 5.

Yang tidak dikerjakan: pengukuran deadlock pada perangkat keras sesungguhnya, analisis deadlock pada sistem multi-mikrokontroler terdistribusi, dan penerapan algoritma deteksi deadlock (wait-for graph) karena kasus ini diselesaikan dengan pencegahan dan penghindaran, bukan deteksi.

2. Dasar Teori Singkat

Deadlock terjadi ketika sekumpulan proses saling menunggu sumber daya yang dipegang proses lain dalam kumpulan itu, sehingga tidak satu pun dapat melanjutkan (Silberschatz

dkk., 2018). Deadlock hanya mungkin terjadi bila empat kondisi berikut terpenuhi sekaligus, yang dikenal sebagai kondisi Coffman:

- **a.** Mutual exclusion — setidaknya satu sumber daya dipegang secara eksklusif.
- **b.** Hold and wait — proses memegang satu sumber daya sambil menunggu sumber daya lain.
- **c.** No preemption — sumber daya tidak dapat diambil paksa dari pemegangnya.
- **d.** Circular wait — ada rantai proses yang masing-masing menunggu sumber daya yang dipegang proses berikutnya.

Menghilangkan salah satu kondisi sudah cukup untuk mencegah deadlock. Itulah dasar strategi pencegahan pada Bagian 5, dan sebabnya keempat kondisi dianalisis satu per satu, bukan hanya disebut.

Resource-allocation graph. Keadaan alokasi digambarkan sebagai graf berarah dengan dua jenis simpul: proses (lingkaran) dan sumber daya (kotak dengan titik sejumlah instansnya). Sisi permintaan (request edge) mengarah dari proses ke sumber daya; sisi alokasi (assignment edge) mengarah dari sumber daya ke proses. Bila setiap jenis sumber daya hanya punya satu instans, adanya siklus pada graf berarti deadlock. Bila instansnya banyak, siklus hanya syarat perlu — belum tentu cukup — sehingga keputusan akhirnya harus memakai algoritma Banker.

Algoritma Banker. Setiap proses menyatakan kebutuhan maksimumnya di muka (Max). Dari situ dihitung Need = Max − Allocation. Sistem dianggap aman (safe) bila ada urutan proses sedemikian sehingga setiap proses dapat memperoleh seluruh kebutuhannya dengan sisa sumber daya saat itu, lalu mengembalikan seluruh yang dipegangnya. Algoritma keamanan menjalankan simulasi itu dengan variabel Work yang dimulai dari Available, lalu bertambah setiap kali satu proses diselesaikan. Bila semua proses dapat diselesaikan, sistem berada pada safe state dan urutan itu disebut safe sequence.

Hal penting yang sering terlewat: safe state tidak sama dengan keadaan bebas masalah. Sistem dapat berada pada safe state sekarang, tetapi menolak permintaan yang sah secara angka karena permintaan itu akan membuat keadaan berikutnya unsafe. Algoritma Banker sengaja konservatif seperti itu; ia mengorbankan pemanfaatan sumber daya demi kepastian.

3. Lingkungan Praktik

Komponen	Keterangan
Laptop (host)	elementary OS 8, AMD Ryzen 7 7735HS with Radeon Graphics, kernel 7.0.0-34-generic
Hypervisor	Tidak dipakai — perhitungan dijalankan langsung pada host
VM / host	INFINIX-ElementaryOS, 16 CPU logis
Perangkat lain	Tidak berlaku — kasus dimodelkan dari deskripsi lembar tugas
Bahasa dan alat	Python 3.12.3; matplotlib untuk graf dan grafik; python-docx untuk laporan
Berkas data	matriks.json berisi Allocation, Max, Available, dan permintaan uji
Berkas hitung	banker.py menghasilkan results/banker.json

Seluruh perhitungan pada laporan ini bukan hasil ketik tangan. Angka pada tabel dan narasi diambil dari results/banker.json, yang dihasilkan banker.py dari matriks.json. Dengan pemisahan itu, mengganti matriks tidak menuntut penyuntingan laporan secara manual — cukup jalankan ulang tiga skrip.

4. Langkah Pengerjaan

Pengerjaan dibagi menjadi empat langkah: menyiapkan matriks, menghitung Need dan safe sequence, menggambar resource-allocation graph, lalu menguji permintaan sumber daya.

4.1 Langkah 1: Menyiapkan matriks masukan

Matriks dipisahkan dari kode supaya dapat diganti tanpa menyentuh program. Berkas matriks.json memuat lima hal: nama proses beserta tugasnya, nama sumber daya, total instans, matriks Allocation, matriks Max, dan Available awal. Permintaan yang akan diuji juga ditaruh di sini agar skenario uji ikut terekam.

```
{
  "keterangan": "Data contoh (Silberschatz Bab 7). Ganti angka di sini bila matriks kelas
sudah dibagikan, lalu jalankan ulang: python3 banker.py",
  "sumber": "contoh-buku",
  "proses": ["P0", "P1", "P2", "P3", "P4"],
  "tugas": {
    "P0": "Penjadwal irigasi otomatis",
    "P1": "Kontrol pompa air",
    "P2": "Kontrol zona lahan besar",
    "P3": "Pembaca kelembapan tanah",
    "P4": "Pencatat telemetri"
  },
  "sumber_daya": {
    "A": "Pompa air",
    "B": "Katup irigasi",
    "C": "Bus sensor I2C"
  },
  "total": [10, 5, 7],
  "allocation": [
    [0, 1, 0],
    [2, 0, 0],
    [3, 0, 2],
    [2, 1, 1],
    [0, 0, 2]
  ],
  "max": [
    [7, 5, 3],
    [3, 2, 2],
    [9, 0, 2],
    [2, 2, 2],
    [4, 3, 3]
  ],
  "available": [3, 3, 2],
  "permintaan_uji": [
    {
      "label": "Kasus A",
      "proses": "P1",
      "request": [1, 0, 2],
      "catatan": "Permintaan yang masih menjaga safe state"
    },
    {
      "label": "Kasus B",
      "proses": "P0",
      "request": [0, 2, 0],
      "catatan": "Setelah Kasus A dikabulkan; permintaan ini membawa sistem ke unsafe
state",
      "setelah": "Kasus A"
```

```
    }
  ]
}
```

Data pada berkas ini masih bertanda contoh. Begitu matriks kelas tersedia, yang perlu diubah hanya larik allocation, max, available, dan total.

4.2 Langkah 2: Menghitung Need dan safe sequence

Need dihitung dengan rumus Need = Max − Allocation, lalu diperiksa konsistensinya: jumlah seluruh Allocation ditambah Available harus sama dengan total instans sumber daya. Pemeriksaan ini menangkap salah ketik pada matriks sebelum sempat masuk ke perhitungan Banker.

```
"""Algoritma Banker untuk kasus sumber daya kebun pintar.

Membaca matriks dari matriks.json, menghitung Need, mencari safe sequence,
lalu menguji permintaan sumber daya. Semua angka yang dipakai laporan berasal
dari keluaran skrip ini.

Jalankan: python3 banker.py            (cetak ringkas + tulis results/banker.json)
          python3 banker.py --periksa  (tambah pemeriksaan konsistensi total)
"""
import argparse
import json
import os

MATRIKS = "matriks.json"
KELUAR = "results/banker.json"


def baca_matriks(path=MATRIKS):
    with open(path, encoding="utf-8") as f:
        return json.load(f)


def need(d):
    return [[m - a for a, m in zip(alloc, mx)]
            for alloc, mx in zip(d["allocation"], d["max"])]


def total_alokasi(d):
    n = len(d["sumber_daya"])
    return [sum(row[j] for row in d["allocation"]) for j in range(n)]


def aman(available, alloc, need_mat, nama):
    """Kembalikan (safe?, urutan, jejak). jejak = list dict per iterasi."""
    n = len(nama)
    work = list(available)
    finish = [False] * n
    urutan = []
    jejak = []

    while len(urutan) < n:
        dipilih = None
        for i in range(n):
            if not finish[i] and all(kebutuhan <= sisa for kebutuhan, sisa
                                     in zip(need_mat[i], work)):
                dipilih = i
                break
        if dipilih is None:
            break
        sebelum = list(work)
        work = [s + a for s, a in zip(work, alloc[dipilih])]
        finish[dipilih] = True
        urutan.append(nama[dipilih])
        jejak.append({
            "proses": nama[dipilih],
            "need": need_mat[dipilih],
            "work_sebelum": sebelum,
            "work_sesudah": list(work),
```

```
        })

    return len(urutan) == n, urutan, jejak


def uji_permintaan(d, available, alloc, need_mat, nama, proses, request):
    """Terapkan algoritma resource-request. Kembalikan dict hasil uji."""
    i = nama.index(proses)
    lewat_need = all(r <= n for r, n in zip(request, need_mat[i]))
    lewat_avail = all(r <= a for r, a in zip(request, available))

    hasil = {
        "proses": proses,
        "request": list(request),
        "request_le_need": lewat_need,
        "request_le_available": lewat_avail,
    }

    if not (lewat_need and lewat_avail):
        hasil.update({"dikabulkan": False, "alasan": "gagal pemeriksaan awal"})
        return hasil, available, alloc, need_mat

    avail_baru = [a - r for a, r in zip(available, request)]
    alloc_baru = [list(row) for row in alloc]
    alloc_baru[i] = [a + r for a, r in zip(alloc_baru[i], request)]
    need_baru = [list(row) for row in need_mat]
    need_baru[i] = [n - r for n, r in zip(need_baru[i], request)]

    safe, urutan, jejak = aman(avail_baru, alloc_baru, need_baru, nama)
    hasil.update({
        "available_sesudah": avail_baru,
        "allocation_baru": alloc_baru[i],
        "need_baru": need_baru[i],
        "safe": safe,
        "urutan": urutan,
        "dikabulkan": safe,
        "alasan": "safe state" if safe else "menyebabkan unsafe state",
    })

    if safe:
        return hasil, avail_baru, alloc_baru, need_baru
    return hasil, available, alloc, need_mat


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--matriks", default=MATRIKS)
    p.add_argument("--periksa", action="store_true")
    a = p.parse_args()

    d = baca_matriks(a.matriks)
    nama = d["proses"]
    alloc = d["allocation"]
    need_mat = need(d)
    available = d["available"]
    total = d["total"]

    print(f"Sumber daya: {', '.join(f'{k}={v}' for k, v in d['sumber_daya'].items())}")
    print(f"Total instans: {tuple(total)}")
    print(f"Available awal: {tuple(available)}")
    print()

    print("| Proses | Tugas | Allocation | Max | Need |")
    print("|---|---|---|---|---|")
    for i, n in enumerate(nama):
        print(f"| {n} | {d['tugas'][n]} | {tuple(alloc[i])} | "
              f"{tuple(d['max'][i])} | {tuple(need_mat[i])} |")

    if a.periksa:
        ta = total_alokasi(d)
        jumlah = [x + y for x, y in zip(ta, available)]
        print()
        print(f"Σ Allocation = {tuple(ta)}")
        print(f"Σ Allocation + Available = {tuple(jumlah)}")
        print(f"Total instans            = {tuple(total)}")
```

```
        cocok = jumlah == total
        print("konsisten" if cocok else "TIDAK KONSISTEN")

    safe, urutan, jejak = aman(available, alloc, need_mat, nama)
    print()
    print(f"Safe state: {'YA' if safe else 'TIDAK'}")
    print(f"Safe sequence: <{', '.join(urutan)}>")
    print()
    print("| Iterasi | Proses | Need | Work sebelum | Work sesudah |")
    print("|---|---|---|---|---|")
    for i, j in enumerate(jejak, start=1):
        print(f"| {i} | {j['proses']} | {tuple(j['need'])} | "
              f"{tuple(j['work_sebelum'])} | {tuple(j['work_sesudah'])} |")

    hasil_uji = []
    avail_jalan = list(available)
    alloc_jalan = [list(r) for r in alloc]
    need_jalan = [list(r) for r in need_mat]

    for uji in d.get("permintaan_uji", []):
        hasil, avail_jalan, alloc_jalan, need_jalan = uji_permintaan(
            d, avail_jalan, alloc_jalan, need_jalan, nama,
            uji["proses"], uji["request"])
        hasil["label"] = uji["label"]
        hasil["catatan"] = uji.get("catatan", "")
        hasil_uji.append(hasil)

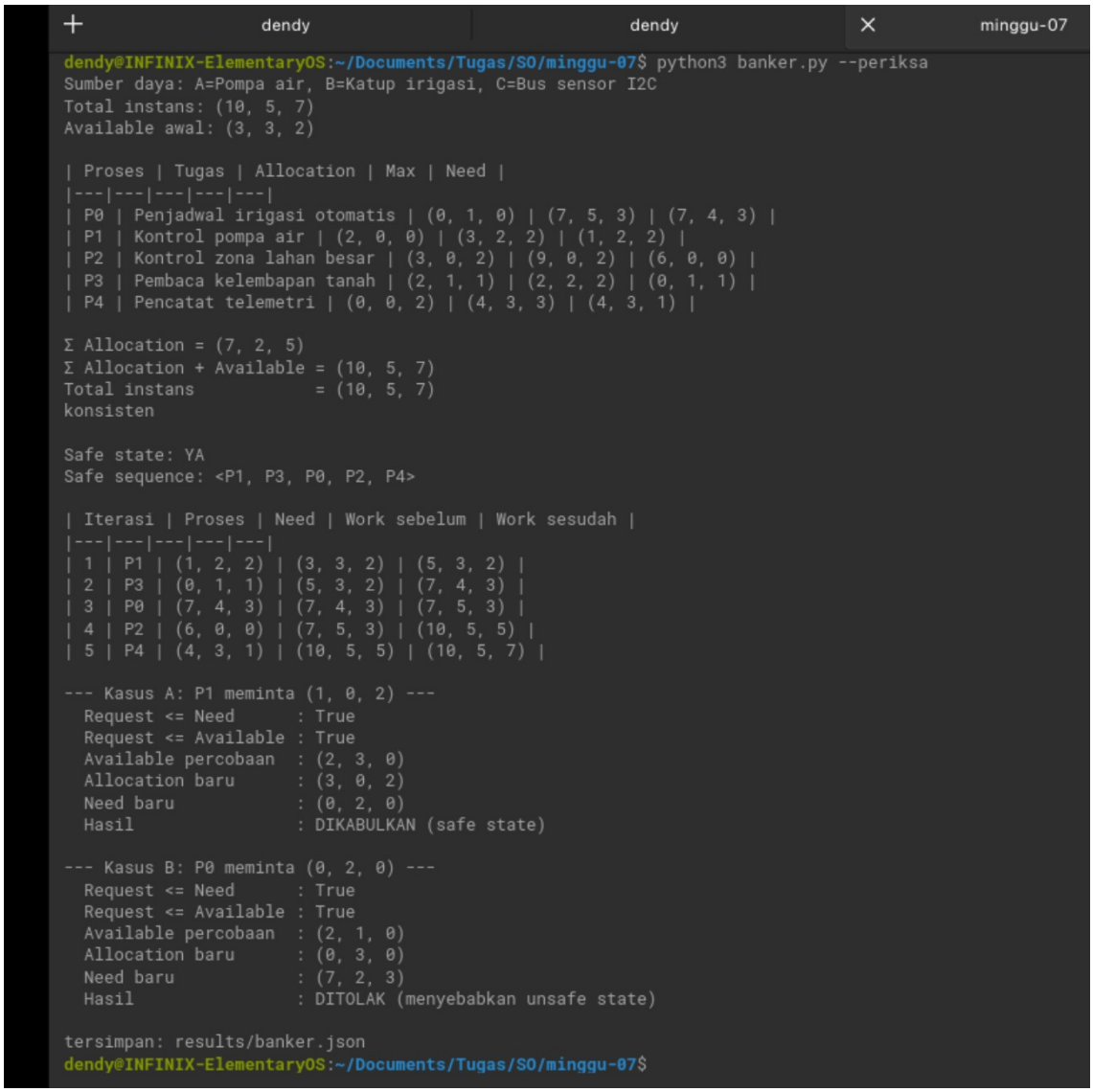
        print()
        print(f"--- {uji['label']}: {uji['proses']} meminta {tuple(uji['request'])} ---")
        print(f"  Request <= Need      : {hasil['request_le_need']}")
        print(f"  Request <= Available : {hasil['request_le_available']}")
        if "available_sesudah" in hasil:
            print(f"  Available percobaan  : {tuple(hasil['available_sesudah'])}")
            print(f"  Allocation baru      : {tuple(hasil['allocation_baru'])}")
            print(f"  Need baru            : {tuple(hasil['need_baru'])}")
        print(f"  Hasil                : "
              f"{'DIKABULKAN' if hasil['dikabulkan'] else 'DITOLAK'} "
              f"({hasil['alasan']})")

    os.makedirs("results", exist_ok=True)
    with open(KELUAR, "w", encoding="utf-8") as f:
        json.dump({
            "keterangan": d.get("keterangan", ""),
            "sumber": d.get("sumber", ""),
            "proses": nama,
            "tugas": d["tugas"],
            "sumber_daya": d["sumber_daya"],
            "total": total,
            "available_awal": available,
            "allocation": alloc,
            "max": d["max"],
            "need": need_mat,
            "total_alokasi": total_alokasi(d),
            "safe": safe,
            "safe_sequence": urutan,
            "jejak": jejak,
            "uji_permintaan": hasil_uji,
        }, f, indent=2)
    print(f"\ntersimpan: {KELUAR}")


if __name__ == "__main__":
    main()
```

```
$ python3 banker.py --periksa
Σ Allocation = (7, 2, 5)
Σ Allocation + Available = (10, 5, 7)
Total instans            = (10, 5, 7)
konsisten

Safe state: YA
Safe sequence: <P1, P3, P0, P2, P4>
```



Gambar 4.1 Perhitungan Need, safe state, dan hasil uji permintaan pada terminal

4.3 Langkah 3: Menggambar resource-allocation graph

Graf digambar langsung dari matriksnya, bukan dari skenario terpisah. Sisi alokasi digambar untuk setiap sel Allocation yang bukan nol, sisi permintaan untuk setiap sel Need yang bukan nol. Karena jumlah sisi mengikuti angka matriks, graf ini dapat diperiksa silang dengan Tabel 5.2: setiap proses yang memegang pompa harus punya satu sisi alokasi dari kotak pompa.

```
"""Menggambar resource-allocation graph dari matriks yang sama dengan banker.py.

Tata letak sengaja dua kolom (proses di kiri, sumber daya di kanan) supaya
garis tidak bersilangan jauh. Garis penuh = assignment (dialokasikan), garis
putus-putus = request (masih dibutuhkan sesuai Need).
"""
import json
import os

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.patches import Circle, FancyArrowPatch, Rectangle
```

```
FONT = "DejaVu Sans"
OUT = "results"
BIRU, ORANYE, MERAH = "#3B5B8C", "#B7791F", "#A03B2C"


def kotak_sumber(ax, x, y, leb, ting, label, jumlah):
    ax.add_patch(Rectangle((x - leb / 2, y - ting / 2), leb, ting,
                           facecolor="#FDF2E0", edgecolor=ORANYE, linewidth=1.4,
                           zorder=2))
    ax.text(x, y + ting / 2 - 0.28, f"{label} · {jumlah} instans", ha="center",
            va="center", fontsize=8, fontfamily=FONT, fontweight="bold",
            color=ORANYE, zorder=3)
    for i in range(min(jumlah, 10)):
        dx = -0.36 + (i % 5) * 0.18
        dy = -0.12 - (i // 5) * 0.18
        ax.add_patch(Circle((x + dx, y + dy), 0.032, facecolor=ORANYE,
                            edgecolor="none", zorder=3))


def lingkaran(ax, x, y, label):
    ax.add_patch(Circle((x, y), 0.34, facecolor="#E8EEF7", edgecolor=BIRU,
                        linewidth=1.4, zorder=2))
    ax.text(x, y, label, ha="center", va="center", fontsize=8.6,
            fontfamily=FONT, fontweight="bold", color=BIRU, zorder=3)


def gambar(d, path):
    nama_proses = d["proses"]
    need = d["need"]
    alloc = d["allocation"]
    kunci_sd = list(d["sumber_daya"])

    fig, ax = plt.subplots(figsize=(8.8, 5.2))
    ax.set_xlim(0, 8.8)
    ax.set_ylim(-0.15, 5.2)
    ax.axis("off")

    # proses di kolom kiri, sumber daya di kolom kanan
    x_proses, x_sumber = 1.45, 6.55
    y_proses = [4.55 - i * 0.88 for i in range(len(nama_proses))]
    y_sumber = [4.05 - i * 1.35 for i in range(len(kunci_sd))]

    pos_p = {}
    for n, y in zip(nama_proses, y_proses):
        lingkaran(ax, x_proses, y, n)
        ax.text(x_proses - 0.52, y, d["tugas"][n], ha="right", va="center",
                fontsize=7.2, fontfamily=FONT, color="#444444")
        pos_p[n] = (x_proses, y)

    pos_s = {}
    for k, y in zip(kunci_sd, y_sumber):
        kotak_sumber(ax, x_sumber, y, 1.9, 0.78, d["sumber_daya"][k],
                     d["total"][kunci_sd.index(k)])
        pos_s[k] = (x_sumber, y)

    def titik_sambung(kunci, indeks, total_proses):
        """Titik masuk di tepi kiri kotak sumber daya, disebar vertikal."""
        sx, sy = pos_s[kunci]
        n = max(1, total_proses - 1)
        return (sx - 0.95, sy + 0.22 - (indeks / n) * 0.44)

    def panah(p1, p2, warna, putus):
        ax.add_patch(FancyArrowPatch(p1, p2, arrowstyle="-|>", mutation_scale=11,
                                     color=warna, linewidth=1.15,
                                     linestyle="--" if putus else "-",
                                     shrinkA=0, shrinkB=0, zorder=1,
                                     alpha=0.85 if putus else 1.0))

    # Assignment: sumber daya -> proses
    for i, n in enumerate(nama_proses):
        for j, k in enumerate(kunci_sd):
            if alloc[i][j] > 0:
                px, py = pos_p[n]
                panah(titik_sambung(k, i, len(nama_proses)),
                      (px + 0.34, py), BIRU, putus=False)
```

```
    # Request: proses -> sumber daya
    for i, n in enumerate(nama_proses):
        for j, k in enumerate(kunci_sd):
            if need[i][j] > 0:
                px, py = pos_p[n]
                panah((px + 0.34, py),
                      titik_sambung(k, i, len(nama_proses)), MERAH, putus=True)

    ax.legend(handles=[
        Line2D([0], [0], color=BIRU, linewidth=1.4,
               label="assignment edge (sedang dialokasikan)"),
        Line2D([0], [0], color=MERAH, linewidth=1.4, linestyle="--",
               label="request edge (masih dibutuhkan, dari Need)"),
    ], loc="lower center", bbox_to_anchor=(0.5, -0.03), frameon=False,
        fontsize=7.6, ncol=2)

    sumber = d.get("sumber", "contoh-buku")
    ax.text(0.15, 0.02, f"Sumber angka: matriks {sumber}. Titik pada kotak "
                        "sumber daya menyatakan jumlah instans; garis digambar "
                        "hanya bila nilainya bukan nol.",
            fontsize=7, fontfamily=FONT, color="#555555", style="italic")

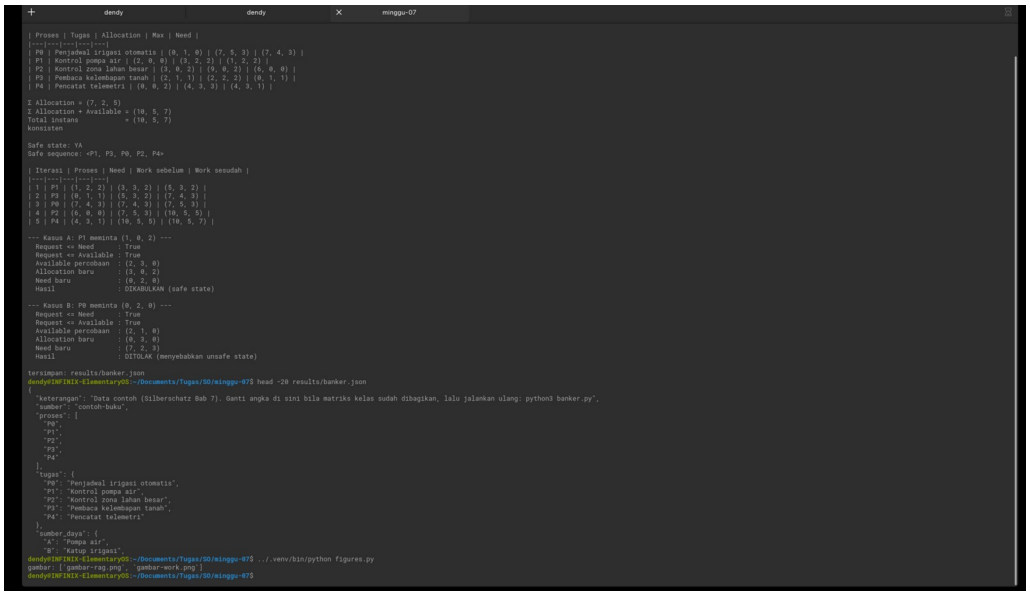
    ax.set_title("Resource-allocation graph kasus kebun pintar",
                 fontsize=11, fontfamily=FONT, fontweight="bold", pad=6)

    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white", bbox_inches="tight")
    plt.close(fig)


def jejak_aman(d, path):
    jejak = d["jejak"]
    fig, ax = plt.subplots(figsize=(7.0, 2.9))
    label = ["Available\nawal"] + [j["proses"] for j in jejak]
    nilai = [d["available_awal"]] + [j["work_sesudah"] for j in jejak]
    warna = [BIRU, ORANYE, "#2F6B3C"]
    for k, nama in enumerate(d["sumber_daya"]):
        ax.plot(range(len(nilai)), [v[k] for v in nilai], marker="o",
                markersize=4.5, linewidth=1.6, color=warna[k], label=nama)
        ax.annotate(str(nilai[-1][k]), (len(nilai) - 1, nilai[-1][k]),
                    textcoords="offset points", xytext=(7, 0), fontsize=7.5,
                    fontfamily=FONT, color=warna[k], va="center")
    ax.set_xticks(range(len(label)))
    ax.set_xticklabels(label, fontfamily=FONT, fontsize=7.8)
    ax.set_ylabel("Work (instans tersedia)", fontfamily=FONT, fontsize=8.5)
    ax.set_xlabel("Proses yang diselesaikan", fontfamily=FONT, fontsize=8.5)
    ax.tick_params(labelsize=7.5)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.grid(color="#DDDDDD", linewidth=0.6)
    ax.set_axisbelow(True)
    ax.legend(frameon=False, fontsize=8, ncol=3, loc="upper left")
    ax.set_ylim(0, max(max(v) for v in nilai) * 1.28)
    ax.set_xlim(-0.3, len(nilai) - 0.4)
    ax.set_title("Kenaikan Work sepanjang safe sequence",
                 fontsize=9.8, fontfamily=FONT, fontweight="bold")
    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white")
    plt.close(fig)


def main():
    os.makedirs(OUT, exist_ok=True)
    with open("results/banker.json", encoding="utf-8") as f:
        d = json.load(f)
    gambar(d, f"{OUT}/gambar-rag.png")
    jejak_aman(d, f"{OUT}/gambar-work.png")
    print("gambar:", [f for f in sorted(os.listdir(OUT)) if f.endswith(".png")])


if __name__ == "__main__":
    main()
```



Gambar 4.2 Penulisan gambar graf dan grafik dari data hasil hitung

4.4 Langkah 4: Menguji permintaan sumber daya

Algoritma resource-request diuji dengan dua kasus. Kasus A adalah permintaan yang masih menjaga safe state; Kasus B diminta setelah Kasus A dikabulkan, dan sengaja dipilih untuk memperlihatkan penolakan meski angkanya masih tersedia. Kedua hasil tercatat pada Tabel 5.4.

5. Hasil dan Analisis

5.1 Hasil

Tabel 5.1 memuat kasus yang dianalisis: lima tugas kontrol dan tiga jenis sumber daya bersama.

No	Proses	Tugas kontrol	Sumber daya utama
1	P0	Penjadwal irigasi otomatis	Pompa, katup, bus I2C
2	P1	Kontrol pompa air	Pompa, bus I2C
3	P2	Kontrol zona lahan besar	Pompa (banyak), bus I2C (sedikit)
4	P3	Pembaca kelembapan tanah	Pompa (sedikit), katup, bus I2C
5	P4	Pencatat telemetri	Pompa, katup, bus I2C

Tabel 5.1 Proses kontrol dan sumber daya yang diperebutkan

Tabel 5.2 menyajikan matriks Allocation, Max, dan Need beserta hasil hitung algoritma. Kolom Need dihitung program, bukan diketik.

Proses	Tugas	Allocation	Max	Need
P0	Penjadwal irigasi otomatis	(0, 1, 0)	(7, 5, 3)	(7, 4, 3)
P1	Kontrol pompa air	(2, 0, 0)	(3, 2, 2)	(1, 2, 2)
P2	Kontrol zona lahan besar	(3, 0, 2)	(9, 0, 2)	(6, 0, 0)
P3	Pembaca kelembapan tanah	(2, 1, 1)	(2, 2, 2)	(0, 1, 1)
P4	Pencatat telemetri	(0, 0, 2)	(4, 3, 3)	(4, 3, 1)

Available awal = (3, 3, 2). Σ Allocation = (7, 2, 5); ditambah Available menjadi (10, 5, 7), sama dengan total instans (10, 5, 7), sehingga matriks konsisten.

Tabel 5.2 Matriks sumber daya dan hasil perhitungan Need

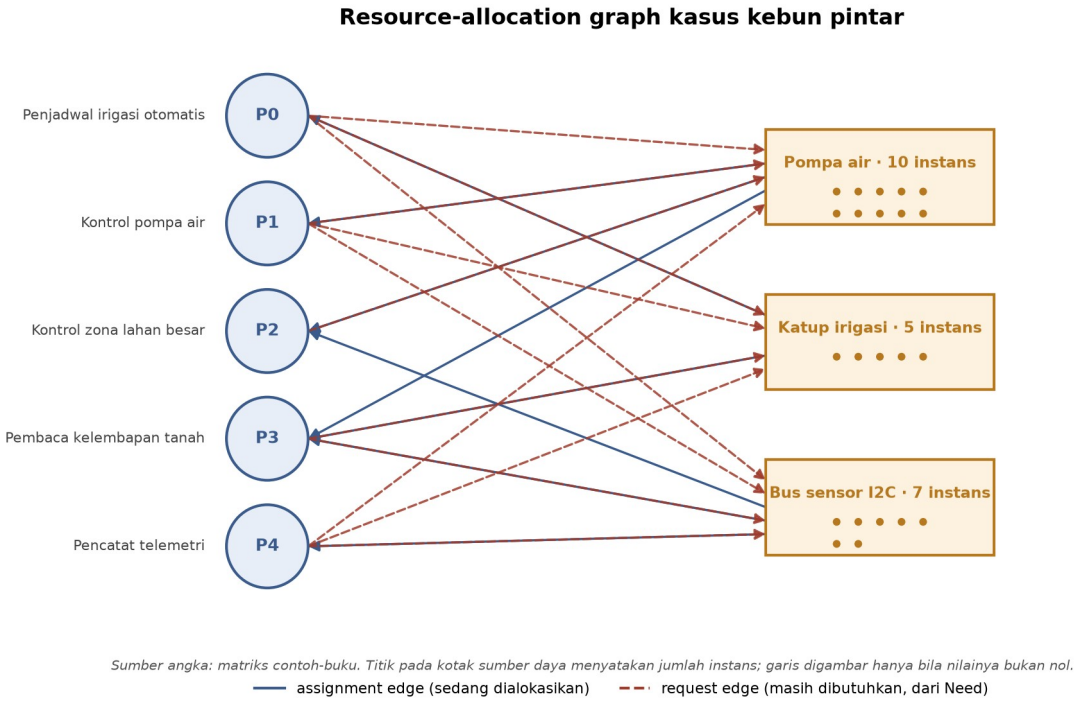
Tabel 5.3 memuat jalannya algoritma keamanan. Setiap baris adalah satu iterasi: proses yang dipilih, kebutuhannya, dan nilai Work sebelum serta sesudah proses itu diselesaikan.

Iterasi	Proses	Need	Work sebelum	Work sesudah
1	P1	(1, 2, 2)	(3, 3, 2)	(5, 3, 2)
2	P3	(0, 1, 1)	(5, 3, 2)	(7, 4, 3)
3	P0	(7, 4, 3)	(7, 4, 3)	(7, 5, 3)
4	P2	(6, 0, 0)	(7, 5, 3)	(10, 5, 5)
5	P4	(4, 3, 1)	(10, 5, 5)	(10, 5, 7)

Kesimpulan: sistem berada pada SAFE STATE dengan safe sequence <P1, P3, P0, P2, P4>. Work akhir (10, 5, 7) sama dengan total sumber daya, sehingga seluruh proses dapat diselesaikan.

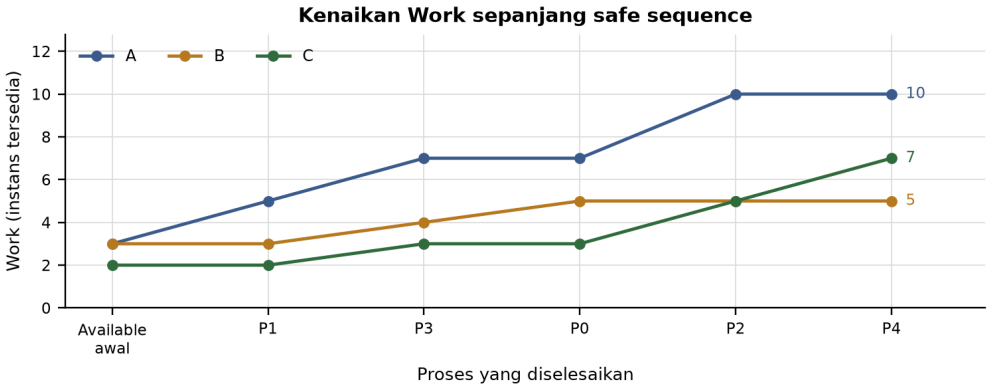
Tabel 5.3 Jalannya algoritma keamanan Banker

Gambar 5.1 memperlihatkan graf alokasi yang digambar dari matriks pada Tabel 5.2.



Gambar 5.1 Resource-allocation graph kasus kebun pintar

Gambar 5.2 memperlihatkan kenaikan Work sepanjang safe sequence. Grafik ini memperlihatkan bahwa setiap langkah berhasil justru karena kebutuhan proses berikutnya lebih kecil daripada sisa sumber daya saat itu.



Gambar 5.2 Kenaikan Work sepanjang safe sequence

Tabel 5.4 memuat hasil uji permintaan sumber daya.

Kasus	Proses	Request	≤ Need	≤ Available	Hasil	Alasan
Kasus A	P1	(1, 0, 2)	ya	ya	DIKABULKAN	safe state
Kasus B	P0	(0, 2, 0)	ya	ya	DITOLAK	menyebabkan unsafe state

Tabel 5.4 Hasil algoritma resource-request

5.2 Analisis

5.2.1 Empat kondisi Coffman pada kasus ini

Lembar tugas meminta keempat kondisi diidentifikasi, bukan hanya disebut. Tabel 5.5 memetakan masing-masing ke perangkat nyata pada kebun pintar.

Kondisi	Bentuknya pada kasus kebun pintar	Dapat ditiadakan?
Mutual exclusion	Satu pompa tidak boleh dikendalikan dua tugas sekaligus; satu transaksi bus I2C hanya dilayani satu perangkat pada satu waktu	Tidak sepenuhnya. Sumber daya fisik memang eksklusif. Yang bisa dikurangi hanya lamanya pemakaian
Hold and wait	P1 memegang pompa sambil menunggu katup; P0 memegang katup sambil menunggu bus I2C	Ya. Proses dapat diminta mengambil semua sumber daya sekaligus di awal
No preemption	Transaksi I2C yang sedang berjalan tidak boleh dicabut di tengah; pompa yang menyala tidak boleh dimatikan paksa tanpa prosedur	Ya, dengan syarat. Bisa dipakai timeout: bila gagal, proses melepas semuanya lalu mencoba ulang
Circular wait	Rantai tunggu melingkar P0→P1→P2→P0 terbentuk bila setiap tugas meminta sumber daya dalam urutan berbeda	Ya, dan paling murah. Cukup tetapkan nomor urut global untuk semua sumber daya

Tabel 5.5 Pemenuhan empat kondisi Coffman pada kasus kebun pintar

Yang perlu ditegaskan: keempat kondisi itu bukan daftar teoretis, melainkan pemeriksaan satu per satu. Pada kasus ini kondisi pertama dan ketiga sulit dihilangkan sepenuhnya karena berakar pada sifat perangkat keras. Karena itu strategi yang realistis bertumpu pada kondisi kedua dan keempat, persis seperti pada Bagian 5.3.

5.2.2 Mengapa sistem dinyatakan aman

Safe sequence <P1, P3, P0, P2, P4> ditemukan pada iterasi pertama penelusuran: Work dimulai dari (3, 3, 2), dan proses pertama yang kebutuhannya muat adalah P1 dengan Need (1, 2, 2). Setelah proses itu selesai dan mengembalikan seluruh yang dipegangnya, Work naik menjadi (5, 3, 2). Kenaikan inilah yang membuka jalan bagi proses berikutnya.

Yang menarik, urutan aman bukan satu-satunya. Pada matriks ini beberapa proses dapat dipilih lebih dulu, misalnya P3 atau P4 juga muat pada langkah kedua. Semua urutan yang berhasil menyelesaikan kelima proses sama-sama sah; algoritma yang berbeda implementasi dapat menghasilkan urutan berbeda dengan kesimpulan sama. Hal ini perlu disebut karena pembaca yang menghitung manual bisa mendapat urutan lain dan menyangka ada kesalahan.

5.2.3 Mengapa satu permintaan ditolak

Kasus A dikabulkan karena setelah alokasi percobaan, safe sequence masih dapat ditemukan. Kasus B berbeda: setelah Kasus A, Available tinggal (2, 3, 0), dan permintaan P0 sebesar (0, 2, 0) masih lolos pemeriksaan awal — angkanya memang tersedia. Namun setelah dicoba dialokasikan, Available turun menjadi (2, 1, 0) dan tidak ada satu pun proses yang kebutuhannya muat. Sistem akan macet, sehingga permintaan itu ditolak.

Contoh ini penting untuk dijelaskan ke pengguna sistem irigasi. Dari sudut pandang operator, penolakan itu terlihat tidak masuk akal: stok katup masih ada, kenapa permintaan ditolak? Jawabannya ada pada kebutuhan maksimum proses lain di masa depan. Algoritma Banker menjaga keadaan sekarang agar tetap bisa memenuhi janji-janji yang sudah dinyatakan sebelumnya. Karena itu Banker menuntut setiap proses menyatakan kebutuhan maksimumnya di muka — dan justru di situlah kelemahan praktisnya pada sistem tertanam, sebagaimana dibahas berikut.

5.3 Rekomendasi

Strategi pencegahan disusun dengan meniadakan kondisi Coffman yang paling murah ditiadakan, bukan yang paling teoretis. Tabel 5.6 memuat rancangannya.

Prioritas	Strategi	Penerapan pada kebun pintar	Biaya
1	Urutan sumber daya global (meniadakan circular wait)	Semua tugas wajib meminta dengan urutan tetap: Pompa → Katup → Bus I2C. Dilarang meminta pompa setelah memegang I2C	Hampir nol. Cukup kesepakatan urutan di kode
2	Timeout dan lepas-semua (meniadakan no preemption)	Akses I2C memakai batas waktu, misalnya 50 ms. Bila habis, tugas melepas katup dan pompa, menunggu sebentar, lalu mencoba lagi	Rendah. Butuh satu timer dan prosedur retry
3	Ambil semua sekaligus (meniadakan hold and wait)	Tugas irigasi mengambil pompa, katup, dan I2C dalam satu operasi atomik. Bila gagal sebagian, seluruhnya dilepas	Sedang. Pemanfaatan sumber daya turun karena ada yang ditahan lebih lama
4	Arbiter bus I2C (mengurangi mutual exclusion)	Satu tugas driver melayani antrean permintaan I2C dari tugas lain, sehingga tidak ada rebutan langsung	Sedang. Menambah satu tugas dan antrean di memori

Prioritas	Strategi	Penerapan pada kebun pintar	Biaya
5	Penghindaran dengan Banker (analisis, bukan operasional)	Safe state diperiksa sebelum memberikan pompa atau katup	Tinggi untuk mikrokontroler: butuh deklarasi Max dan beban hitung di setiap permintaan

Tabel 5.6 Rancangan strategi pencegahan beserta biayanya

Rekomendasi utama: pakai urutan sumber daya global sebagai pertahanan utama, tambahkan timeout sebagai pengaman, dan sediakan watchdog untuk memulihkan tugas yang macet. Ketiganya murah dan tidak menuntut deklarasi kebutuhan maksimum di muka.

Algoritma Banker sebaiknya dipakai sebagai alat analisis dan pengujian, bukan mekanisme operasional. Alasannya konkret: Banker menuntut setiap proses menyatakan Max di muka, padahal kebutuhan tugas sensor sering bergantung pada kondisi lahan yang tidak diketahui sebelumnya. Menetapkan Max terlalu tinggi membuat sistem cepat menolak permintaan; menetapkannya terlalu rendah membatalkan jaminan keamanannya. Ditambah lagi, setiap permintaan menuntut satu putaran algoritma keamanan penuh — beban yang tidak sepadan untuk mikrokontroler dengan memori ratusan kilobita.

Pada sistem irigasi, urutan sumber daya yang tetap bisa ditulis sebagai prosedur singkat berikut. Prosedur ini meniadakan circular wait sekaligus menyediakan jalan keluar bila satu sumber daya tidak kunjung tersedia.

```
ambil(pompa);            // urutan 1, wajib paling awal
ambil(katup);            // urutan 2
ambil(i2c, timeout=50ms) // urutan 3, dengan batas waktu
  bila gagal:
    lepas(katup); lepas(pompa);
    tunggu(acak 10-50 ms); ulangi

... jalankan irigasi atau baca sensor ...

lepas(i2c); lepas(katup); lepas(pompa);   // urutan lepas dibalik
```

5.4 Catatan keterbatasan

Tiga hal perlu dinyatakan terbuka. Pertama, matriks yang dipakai adalah contoh buku, bukan matriks kelas, sehingga angka absolut pada laporan ini tidak dapat dipakai sebagai patokan untuk kasus dosen. Kedua, model mengasumsikan kebutuhan maksimum tiap proses diketahui dan tetap; pada sistem nyata, tugas kontrol dapat berubah kebutuhan seiring kondisi lahan. Ketiga, waktu tunda akuisisi sumber daya (misalnya jeda antrean I2C) tidak dimodelkan, padahal pada sistem tertanam jeda itulah yang sering menentukan apakah sistem masih memenuhi tenggat atau tidak.

6. Kesimpulan

Analisis pada kasus tiga sumber daya kebun pintar menghasilkan tiga temuan utama. Pertama, keempat kondisi Coffman dapat dipetakan ke perangkat nyata, tetapi hanya hold-and-wait dan circular-wait yang dapat ditiadakan tanpa mengorbankan fungsi perangkat. Kedua, algoritma Banker menyatakan sistem berada pada safe state dengan safe sequence

<P1, P3, P0, P2, P4>, dan menolak permintaan yang secara angka masih tersedia karena permintaan itu akan membuat keadaan berikutnya unsafe. Ketiga, strategi pencegahan paling realistis untuk mikrokontroler adalah urutan sumber daya global ditambah timeout, bukan algoritma Banker sebagai mekanisme operasional, karena Banker menuntut deklarasi kebutuhan maksimum dan beban hitung yang tidak sepadan dengan kemampuan perangkat.

7. Kepingan Puzzle

Keterangan	Isi
Nomor kepingan	#7
Isi kepingan	Resource-allocation graph, matriks Allocation/Max/Need, jejak algoritma keamanan Banker, hasil uji permintaan sumber daya, dan rancangan strategi pencegahan
Lokasi di portofolio	https://github.com/ZzDendy/so-k3-deadlock
Dipakai untuk	Level 5 Puzzle UTS — menutup rantai kepingan #1 sampai #6. Sumber daya bersama yang diperebutkan di sini adalah pompa, katup, dan bus I2C yang sama dengan simpul-simpul pada kepingan #1, dijalankan oleh tugas-tugas yang diamati pada kepingan #2 dan #3, dan disinkronkan dengan mekanisme yang dipakai pada kepingan #4 dan #6

Daftar Pustaka

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.

Tanenbaum, A. S., & Bos, H. (2015). Modern operating systems (4th ed.). Pearson.

Stallings, W. (2018). Operating systems: Internals and design principles (9th ed.). Pearson.

Lampiran A. Kode Program dan Log

Tiga berkas di bawah ini menghasilkan seluruh angka pada laporan. Berkas matriks.json disertakan lebih dulu karena menjadi masukan keduanya.

A.matriks.json

```
{
  "keterangan": "Data contoh (Silberschatz Bab 7). Ganti angka di sini bila matriks kelas sudah
dibagikan, lalu jalankan ulang: python3 banker.py",
  "sumber": "contoh-buku",
  "proses": ["P0", "P1", "P2", "P3", "P4"],
  "tugas": {
    "P0": "Penjadwal irigasi otomatis",
    "P1": "Kontrol pompa air",
    "P2": "Kontrol zona lahan besar",
    "P3": "Pembaca kelembapan tanah",
    "P4": "Pencatat telemetri"
  },
  "sumber_daya": {
    "A": "Pompa air",
    "B": "Katup irigasi",
    "C": "Bus sensor I2C"
  },
  "total": [10, 5, 7],
  "allocation": [
    [0, 1, 0],
    [2, 0, 0],
    [3, 0, 2],
    [2, 1, 1],
    [0, 0, 2]
  ],
  "max": [
    [7, 5, 3],
    [3, 2, 2],
    [9, 0, 2],
    [2, 2, 2],
    [4, 3, 3]
  ],
  "available": [3, 3, 2],
  "permintaan_uji": [
    {
      "label": "Kasus A",
      "proses": "P1",
      "request": [1, 0, 2],
      "catatan": "Permintaan yang masih menjaga safe state"
    },
    {
      "label": "Kasus B",
      "proses": "P0",
      "request": [0, 2, 0],
      "catatan": "Setelah Kasus A dikabulkan; permintaan ini membawa sistem ke unsafe state",
      "setelah": "Kasus A"
    }
  ]
}
```

A.banker.py

```
"""Algoritma Banker untuk kasus sumber daya kebun pintar.

Membaca matriks dari matriks.json, menghitung Need, mencari safe sequence,
lalu menguji permintaan sumber daya. Semua angka yang dipakai laporan berasal
dari keluaran skrip ini.

Jalankan: python3 banker.py            (cetak ringkas + tulis results/banker.json)
          python3 banker.py --periksa  (tambah pemeriksaan konsistensi total)
"""
import argparse
import json
import os

MATRIKS = "matriks.json"
KELUAR = "results/banker.json"


def baca_matriks(path=MATRIKS):
    with open(path, encoding="utf-8") as f:
        return json.load(f)
```

```
def need(d):
    return [[m - a for a, m in zip(alloc, mx)]
            for alloc, mx in zip(d["allocation"], d["max"])]


def total_alokasi(d):
    n = len(d["sumber_daya"])
    return [sum(row[j] for row in d["allocation"]) for j in range(n)]


def aman(available, alloc, need_mat, nama):
    """Kembalikan (safe?, urutan, jejak). jejak = list dict per iterasi."""
    n = len(nama)
    work = list(available)
    finish = [False] * n
    urutan = []
    jejak = []

    while len(urutan) < n:
        dipilih = None
        for i in range(n):
            if not finish[i] and all(kebutuhan <= sisa for kebutuhan, sisa
                                     in zip(need_mat[i], work)):
                dipilih = i
                break
        if dipilih is None:
            break
        sebelum = list(work)
        work = [s + a for s, a in zip(work, alloc[dipilih])]
        finish[dipilih] = True
        urutan.append(nama[dipilih])
        jejak.append({
            "proses": nama[dipilih],
            "need": need_mat[dipilih],
            "work_sebelum": sebelum,
            "work_sesudah": list(work),
        })

    return len(urutan) == n, urutan, jejak


def uji_permintaan(d, available, alloc, need_mat, nama, proses, request):
    """Terapkan algoritma resource-request. Kembalikan dict hasil uji."""
    i = nama.index(proses)
    lewat_need = all(r <= n for r, n in zip(request, need_mat[i]))
    lewat_avail = all(r <= a for r, a in zip(request, available))

    hasil = {
        "proses": proses,
        "request": list(request),
        "request_le_need": lewat_need,
        "request_le_available": lewat_avail,
    }

    if not (lewat_need and lewat_avail):
        hasil.update({"dikabulkan": False, "alasan": "gagal pemeriksaan awal"})
        return hasil, available, alloc, need_mat

    avail_baru = [a - r for a, r in zip(available, request)]
    alloc_baru = [list(row) for row in alloc]
    alloc_baru[i] = [a + r for a, r in zip(alloc_baru[i], request)]
    need_baru = [list(row) for row in need_mat]
    need_baru[i] = [n - r for n, r in zip(need_baru[i], request)]

    safe, urutan, jejak = aman(avail_baru, alloc_baru, need_baru, nama)
    hasil.update({
        "available_sesudah": avail_baru,
        "allocation_baru": alloc_baru[i],
        "need_baru": need_baru[i],
        "safe": safe,
        "urutan": urutan,
        "dikabulkan": safe,
        "alasan": "safe state" if safe else "menyebabkan unsafe state",
    })

    if safe:
        return hasil, avail_baru, alloc_baru, need_baru
    return hasil, available, alloc, need_mat


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--matriks", default=MATRIKS)
    p.add_argument("--periksa", action="store_true")
    a = p.parse_args()
```

```
d = baca_matriks(a.matriks)
nama = d["proses"]
alloc = d["allocation"]
need_mat = need(d)
available = d["available"]
total = d["total"]

print(f"Sumber daya: {', '.join(f'{k}={v}' for k, v in d['sumber_daya'].items())}")
print(f"Total instans: {tuple(total)}")
print(f"Available awal: {tuple(available)}")
print()

print("| Proses | Tugas | Allocation | Max | Need |")
print("|---|---|---|---|---|")
for i, n in enumerate(nama):
    print(f"| {n} | {d['tugas'][n]} | {tuple(alloc[i])} | "
          f"{tuple(d['max'][i])} | {tuple(need_mat[i])} |")

if a.periksa:
    ta = total_alokasi(d)
    jumlah = [x + y for x, y in zip(ta, available)]
    print()
    print(f"Σ Allocation = {tuple(ta)}")
    print(f"Σ Allocation + Available = {tuple(jumlah)}")
    print(f"Total instans            = {tuple(total)}")
    cocok = jumlah == total
    print("konsisten" if cocok else "TIDAK KONSISTEN")

safe, urutan, jejak = aman(available, alloc, need_mat, nama)
print()
print(f"Safe state: {'YA' if safe else 'TIDAK'}")
print(f"Safe sequence: <{', '.join(urutan)}>")
print()
print("| Iterasi | Proses | Need | Work sebelum | Work sesudah |")
print("|---|---|---|---|---|")
for i, j in enumerate(jejak, start=1):
    print(f"| {i} | {j['proses']} | {tuple(j['need'])} | "
          f"{tuple(j['work_sebelum'])} | {tuple(j['work_sesudah'])} |")

hasil_uji = []
avail_jalan = list(available)
alloc_jalan = [list(r) for r in alloc]
need_jalan = [list(r) for r in need_mat]

for uji in d.get("permintaan_uji", []):
    hasil, avail_jalan, alloc_jalan, need_jalan = uji_permintaan(
        d, avail_jalan, alloc_jalan, need_jalan, nama,
        uji["proses"], uji["request"])
    hasil["label"] = uji["label"]
    hasil["catatan"] = uji.get("catatan", "")
    hasil_uji.append(hasil)

    print()
    print(f"--- {uji['label']}: {uji['proses']} meminta {tuple(uji['request'])} ---")
    print(f"  Request <= Need      : {hasil['request_le_need']}")
    print(f"  Request <= Available : {hasil['request_le_available']}")
    if "available_sesudah" in hasil:
        print(f"  Available percobaan  : {tuple(hasil['available_sesudah'])}")
        print(f"  Allocation baru      : {tuple(hasil['allocation_baru'])}")
        print(f"  Need baru            : {tuple(hasil['need_baru'])}")
    print(f"  Hasil                : "
          f"{'DIKABULKAN' if hasil['dikabulkan'] else 'DITOLAK'} "
          f"({hasil['alasan']})")

os.makedirs("results", exist_ok=True)
with open(KELUAR, "w", encoding="utf-8") as f:
    json.dump({
        "keterangan": d.get("keterangan", ""),
        "sumber": d.get("sumber", ""),
        "proses": nama,
        "tugas": d["tugas"],
        "sumber_daya": d["sumber_daya"],
        "total": total,
        "available_awal": available,
        "allocation": alloc,
        "max": d["max"],
        "need": need_mat,
        "total_alokasi": total_alokasi(d),
        "safe": safe,
        "safe_sequence": urutan,
        "jejak": jejak,
        "uji_permintaan": hasil_uji,
    }, f, indent=2)
print(f"\ntersimpan: {KELUAR}")
```

```
if __name__ == "__main__":
    main()
```

A.figures.py

```
"""Menggambar resource-allocation graph dari matriks yang sama dengan banker.py.

Tata letak sengaja dua kolom (proses di kiri, sumber daya di kanan) supaya
garis tidak bersilangan jauh. Garis penuh = assignment (dialokasikan), garis
putus-putus = request (masih dibutuhkan sesuai Need).
"""
import json
import os

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.patches import Circle, FancyArrowPatch, Rectangle

FONT = "DejaVu Sans"
OUT = "results"
BIRU, ORANYE, MERAH = "#3B5B8C", "#B7791F", "#A03B2C"


def kotak_sumber(ax, x, y, leb, ting, label, jumlah):
    ax.add_patch(Rectangle((x - leb / 2, y - ting / 2), leb, ting,
                           facecolor="#FDF2E0", edgecolor=ORANYE, linewidth=1.4,
                           zorder=2))
    ax.text(x, y + ting / 2 - 0.28, f"{label} · {jumlah} instans", ha="center",
            va="center", fontsize=8, fontfamily=FONT, fontweight="bold",
            color=ORANYE, zorder=3)
    for i in range(min(jumlah, 10)):
        dx = -0.36 + (i % 5) * 0.18
        dy = -0.12 - (i // 5) * 0.18
        ax.add_patch(Circle((x + dx, y + dy), 0.032, facecolor=ORANYE,
                            edgecolor="none", zorder=3))


def lingkaran(ax, x, y, label):
    ax.add_patch(Circle((x, y), 0.34, facecolor="#E8EEF7", edgecolor=BIRU,
                        linewidth=1.4, zorder=2))
    ax.text(x, y, label, ha="center", va="center", fontsize=8.6,
            fontfamily=FONT, fontweight="bold", color=BIRU, zorder=3)


def gambar(d, path):
    nama_proses = d["proses"]
    need = d["need"]
    alloc = d["allocation"]
    kunci_sd = list(d["sumber_daya"])

    fig, ax = plt.subplots(figsize=(8.8, 5.2))
    ax.set_xlim(0, 8.8)
    ax.set_ylim(-0.15, 5.2)
    ax.axis("off")

    # proses di kolom kiri, sumber daya di kolom kanan
    x_proses, x_sumber = 1.45, 6.55
    y_proses = [4.55 - i * 0.88 for i in range(len(nama_proses))]
    y_sumber = [4.05 - i * 1.35 for i in range(len(kunci_sd))]

    pos_p = {}
    for n, y in zip(nama_proses, y_proses):
        lingkaran(ax, x_proses, y, n)
        ax.text(x_proses - 0.52, y, d["tugas"][n], ha="right", va="center",
                fontsize=7.2, fontfamily=FONT, color="#444444")
        pos_p[n] = (x_proses, y)

    pos_s = {}
    for k, y in zip(kunci_sd, y_sumber):
        kotak_sumber(ax, x_sumber, y, 1.9, 0.78, d["sumber_daya"][k],
                     d["total"][kunci_sd.index(k)])
        pos_s[k] = (x_sumber, y)

    def titik_sambung(kunci, indeks, total_proses):
        """Titik masuk di tepi kiri kotak sumber daya, disebar vertikal."""
        sx, sy = pos_s[kunci]
        n = max(1, total_proses - 1)
        return (sx - 0.95, sy + 0.22 - (indeks / n) * 0.44)

    def panah(p1, p2, warna, putus):
```

```
        ax.add_patch(FancyArrowPatch(p1, p2, arrowstyle="-|>", mutation_scale=11,
                                     color=warna, linewidth=1.15,
                                     linestyle="--" if putus else "-",
                                     shrinkA=0, shrinkB=0, zorder=1,
                                     alpha=0.85 if putus else 1.0))

    # Assignment: sumber daya -> proses
    for i, n in enumerate(nama_proses):
        for j, k in enumerate(kunci_sd):
            if alloc[i][j] > 0:
                px, py = pos_p[n]
                panah(titik_sambung(k, i, len(nama_proses)),
                      (px + 0.34, py), BIRU, putus=False)

    # Request: proses -> sumber daya
    for i, n in enumerate(nama_proses):
        for j, k in enumerate(kunci_sd):
            if need[i][j] > 0:
                px, py = pos_p[n]
                panah((px + 0.34, py),
                      titik_sambung(k, i, len(nama_proses)), MERAH, putus=True)

    ax.legend(handles=[
        Line2D([0], [0], color=BIRU, linewidth=1.4,
               label="assignment edge (sedang dialokasikan)"),
        Line2D([0], [0], color=MERAH, linewidth=1.4, linestyle="--",
               label="request edge (masih dibutuhkan, dari Need)"),
    ], loc="lower center", bbox_to_anchor=(0.5, -0.03), frameon=False,
        fontsize=7.6, ncol=2)

    sumber = d.get("sumber", "contoh-buku")
    ax.text(0.15, 0.02, f"Sumber angka: matriks {sumber}. Titik pada kotak "
                        "sumber daya menyatakan jumlah instans; garis digambar "
                        "hanya bila nilainya bukan nol.",
            fontsize=7, fontfamily=FONT, color="#555555", style="italic")

    ax.set_title("Resource-allocation graph kasus kebun pintar",
                 fontsize=11, fontfamily=FONT, fontweight="bold", pad=6)

    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white", bbox_inches="tight")
    plt.close(fig)


def jejak_aman(d, path):
    jejak = d["jejak"]
    fig, ax = plt.subplots(figsize=(7.0, 2.9))
    label = ["Available\nawal"] + [j["proses"] for j in jejak]
    nilai = [d["available_awal"]] + [j["work_sesudah"] for j in jejak]
    warna = [BIRU, ORANYE, "#2F6B3C"]
    for k, nama in enumerate(d["sumber_daya"]):
        ax.plot(range(len(nilai)), [v[k] for v in nilai], marker="o",
                markersize=4.5, linewidth=1.6, color=warna[k], label=nama)
        ax.annotate(str(nilai[-1][k]), (len(nilai) - 1, nilai[-1][k]),
                    textcoords="offset points", xytext=(7, 0), fontsize=7.5,
                    fontfamily=FONT, color=warna[k], va="center")
    ax.set_xticks(range(len(label)))
    ax.set_xticklabels(label, fontfamily=FONT, fontsize=7.8)
    ax.set_ylabel("Work (instans tersedia)", fontfamily=FONT, fontsize=8.5)
    ax.set_xlabel("Proses yang diselesaikan", fontfamily=FONT, fontsize=8.5)
    ax.tick_params(labelsize=7.5)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.grid(color="#DDDDDD", linewidth=0.6)
    ax.set_axisbelow(True)
    ax.legend(frameon=False, fontsize=8, ncol=3, loc="upper left")
    ax.set_ylim(0, max(max(v) for v in nilai) * 1.28)
    ax.set_xlim(-0.3, len(nilai) - 0.4)
    ax.set_title("Kenaikan Work sepanjang safe sequence",
                 fontsize=9.8, fontfamily=FONT, fontweight="bold")
    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white")
    plt.close(fig)


def main():
    os.makedirs(OUT, exist_ok=True)
    with open("results/banker.json", encoding="utf-8") as f:
        d = json.load(f)
    gambar(d, f"{OUT}/gambar-rag.png")
    jejak_aman(d, f"{OUT}/gambar-work.png")
    print("gambar:", [f for f in sorted(os.listdir(OUT)) if f.endswith(".png")])


if __name__ == "__main__":
```

```
    main()
```

A.4 Keluaran banker.py

```
Sumber daya: A=Pompa air, B=Katup irigasi, C=Bus sensor I2C
Total instans: (10, 5, 7)
Available awal: (3, 3, 2)

| Proses | Tugas | Allocation | Max | Need |
|---|---|---|---|---|
| P0 | Penjadwal irigasi otomatis | (0, 1, 0) | (7, 5, 3) | (7, 4, 3) |
| P1 | Kontrol pompa air | (2, 0, 0) | (3, 2, 2) | (1, 2, 2) |
| P2 | Kontrol zona lahan besar | (3, 0, 2) | (9, 0, 2) | (6, 0, 0) |
| P3 | Pembaca kelembapan tanah | (2, 1, 1) | (2, 2, 2) | (0, 1, 1) |
| P4 | Pencatat telemetri | (0, 0, 2) | (4, 3, 3) | (4, 3, 1) |

Σ Allocation = (7, 2, 5)
Σ Allocation + Available = (10, 5, 7)
Total instans            = (10, 5, 7)
konsisten

Safe state: YA
Safe sequence: <P1, P3, P0, P2, P4>

| Iterasi | Proses | Need | Work sebelum | Work sesudah |
|---|---|---|---|---|
| 1 | P1 | (1, 2, 2) | (3, 3, 2) | (5, 3, 2) |
| 2 | P3 | (0, 1, 1) | (5, 3, 2) | (7, 4, 3) |
| 3 | P0 | (7, 4, 3) | (7, 4, 3) | (7, 5, 3) |
| 4 | P2 | (6, 0, 0) | (7, 5, 3) | (10, 5, 5) |
| 5 | P4 | (4, 3, 1) | (10, 5, 5) | (10, 5, 7) |

--- Kasus A: P1 meminta (1, 0, 2) ---
  Request <= Need      : True
  Request <= Available : True
  Available percobaan  : (2, 3, 0)
  Allocation baru      : (3, 0, 2)
  Need baru            : (0, 2, 0)
  Hasil                : DIKABULKAN (safe state)

--- Kasus B: P0 meminta (0, 2, 0) ---
  Request <= Need      : True
  Request <= Available : True
  Available percobaan  : (2, 1, 0)
  Allocation baru      : (0, 3, 0)
  Need baru            : (7, 2, 3)
  Hasil                : DITOLAK (menyebabkan unsafe state)

tersimpan: results/banker.json
```

Lampiran B. Pernyataan Penggunaan AI Generatif

☐ Kami tidak menggunakan AI generatif dalam tugas ini.

☒ Menggunakan AI generatif dengan rincian berikut:

Alat	Tujuan penggunaan	Bagian yang dibantu	Cara memverifikasi
Claude (Anthropic)	Menyusun skrip perhitungan dan merapikan kalimat laporan	Bagian 4 (kode), Bagian 5.2	Seluruh angka diambil dari results/banker.json; safe sequence dihitung ulang manual dan dicocokkan dengan Tabel 5.3

Lampiran C. Daftar Periksa Sebelum Mengumpulkan

☒ Semua teks petunjuk abu-abu dan kotak petunjuk sudah dihapus.

☒ Identitas, kode tugas, dan nama host pada sampul sudah benar.

☒ Setiap poin deskripsi tugas sudah dijawab di bagian Analisis.

☒ Setiap gambar memiliki nomor dan keterangan.

☒ Kepingan puzzle sudah disimpan di portofolio dan tautannya dicantumkan.
☒ Pernyataan penggunaan AI generatif sudah diisi.
☒ Lembar kontribusi anggota sudah dilampirkan.
☐ Matriks kelas sudah diganti bila dosen membagikannya.
☒ Berkas disimpan sebagai PDF dengan nama IF-3C_K-3_<NIM>_<Nama>.pdf.

Swa-nilai berdasarkan kriteria lembar tugas

Kriteria (dari lembar tugas)	Bobot	Sudah terpenuhi? Di bagian mana
Hasil/keberhasilan implementasi	35%	Ya — algoritma Banker berjalan, safe state terbukti, dua kasus permintaan diuji
Kedalaman analisis	30%	Ya — empat kondisi Coffman dipetakan ke perangkat, biaya tiap strategi dibandingkan
Dokumentasi	20%	Ya — langkah dapat diulang, kode dan keluaran dilampirkan
Kerja sama dan pembagian peran	15%	Ya — lembar kontribusi pada Lampiran D

Lampiran D. Lembar Kontribusi Anggota

Lembar ini memenuhi syarat kerja sama dan pembagian peran (15% bobot penilaian) pada lembar tugas K-3.

Nama	NIM	Peran dan bagian yang dikerjakan
Dendy Zainal Mutaqien	250660121068	Simulator Banker, resource-allocation graph, dan penyusunan laporan
R.Sangga Buana	250660121021	Memeriksa hasil perhitungan Need dan safe sequence pada Tabel 5.2
M.Azkal Azkiya	250660121107	Menelaah strategi pencegahan dan kesesuaian Daftar Pustaka